

SSL-TLS

WHAT IS SSL/TLS?

Tôi đoán rằng nhiều bạn đang xem video này đã biết về HTTPS, và một số bạn thậm chí đã từng cấu hình SSL/TLS cho web server của mình. Nhưng có bao nhiêu người trong số các bạn thực sự hiểu sâu về cách SSL/TLS hoạt động?

Bạn có biết thực sự điều gì xảy ra trong quá trình TLS handshake không?

Tại sao chúng ta lại cần handshake?

TLS sử dụng những thuật toán mật mã nào để bảo vệ dữ liệu?

Vì sao chúng ta cần digital certificate?

Tại sao certificate lại phải được ký bởi Certificate Authority (CA)?

Digital signature là gì và nó được tạo ra như thế nào?

Có rất nhiều câu hỏi như vậy, và tôi không muốn chỉ nói sơ qua bề mặt. Vì thế, video này sẽ là một video rất chi tiết, giúp bạn hiểu toàn bộ mọi thứ về SSL/TLS — một thành phần nền tảng cực kỳ quan trọng của hệ thống bảo mật trên Internet.

SSL/TLS là gì?

SSL là viết tắt của Secure Socket Layer. Đây là tiền thân của TLS.

Còn TLS là viết tắt của Transport Layer Security, một giao thức mật mã cung cấp khả năng giao tiếp an toàn trên mạng máy tính.

Hãy cùng nhìn lại một chút lịch sử của SSL và TLS.

SSL ban đầu được phát triển bởi Netscape và lần đầu tiên được công bố vào năm 1995. Phiên bản SSL 1.0 chưa bao giờ được phát hành công khai do tồn tại những lỗ hổng bảo mật nghiêm trọng.

Đến năm 1996, SSL 3.0 được công bố như một phiên bản thiết kế lại hoàn toàn của giao thức.

Ba năm sau, TLS 1.0 lần đầu tiên được định nghĩa trong RFC 2246 bởi IETF, như một bản nâng cấp từ SSL 3.0.

Phải mất 7 năm sau đó, giao thức mới được nâng cấp lên TLS 1.1 vào năm 2006. Tiếp theo là TLS 1.2, được phát hành vào năm 2008.

Cuối cùng, sau 10 năm phát triển, chúng ta có TLS 1.3 với những cải tiến rất lớn.

Vậy hiện nay những phiên bản SSL/TLS nào còn tồn tại?

SSL 2.0 đã bị khai tử (deprecated) vào năm 2011.

SSL 3.0 bị khai tử vào năm 2015.

Và gần đây hơn, vào tháng 3 năm 2020, TLS 1.0 và TLS 1.1 cũng chính thức bị loại bỏ.

Điều đó có nghĩa là hiện tại chỉ còn TLS 1.2 và TLS 1.3 vẫn đang được sử dụng.

Bây giờ, hãy xem TLS được sử dụng ở đâu.

Đầu tiên, TLS được sử dụng rất rộng rãi trên web. Tất cả các website bạn truy cập với HTTPS đều được bảo mật bởi TLS, hay chúng ta thường nói là HTTP over TLS.

Tương tự, email với giao thức SMTPS thực chất là SMTP kết hợp với TLS.

FTPS, giao thức truyền file an toàn, cũng là FTP cộng với TLS.

Ngoài ra, TLS còn được sử dụng trong rất nhiều ứng dụng khác mà tôi không có đủ thời gian để kể hết.

Vậy tại sao chúng ta cần TLS? Tại sao nó lại quan trọng đến vậy?

Bởi vì TLS mang lại cho chúng ta 3 yếu tố cốt lõi.

Thứ nhất là Authentication (xác thực).

TLS xác minh danh tính của các bên giao tiếp, thường là client và server. Với sự trợ giúp của mật mã bất đối xứng, TLS đảm bảo rằng chúng ta đang truy cập website thật, chứ không phải website giả mạo.

Thứ hai là Confidentiality (tính bảo mật).

TLS bảo vệ dữ liệu được trao đổi khỏi việc truy cập trái phép bằng cách mã hóa dữ liệu với các thuật toán mã hóa đối xứng.

Thứ ba là Integrity (tính toàn vẹn).

TLS có thể phát hiện mọi sự thay đổi dữ liệu trong quá trình truyền tải bằng cách kiểm tra Message Authentication Code (MAC), khái niệm mà chúng ta sẽ tìm hiểu ngay sau đây.

HOW TLS WORKS?

Bây giờ, câu hỏi tiếp theo là: TLS hoạt động như thế nào?

Về cơ bản, TLS bao gồm 2 giai đoạn, hay có thể xem là 2 giao thức.

Giai đoạn đầu tiên là Handshake Protocol.

Trong giai đoạn này, client và server sẽ:

- Thương lượng phiên bản giao thức
- Lựa chọn thuật toán mật mã, hay còn gọi là cipher suite
- Xác thực lẫn nhau bằng mật mã bất đối xứng

- Và thiết lập một khóa bí mật dùng chung, khóa này sẽ được sử dụng cho mã hóa đối xứng ở giai đoạn tiếp theo

Vì vậy, mục đích chính của handshake là xác thực và trao đổi khóa.

Giai đoạn thứ hai là Record Protocol.

Trong giai đoạn này, tất cả các message gửi đi sẽ được mã hóa bằng khóa bí mật dùng chung đã được thiết lập trong quá trình handshake.

Sau đó, các message đã được mã hóa sẽ được truyền sang phía còn lại. Ở bên nhận, chúng sẽ được kiểm tra xem có bị chỉnh sửa trong quá trình truyền hay không. Nếu không có thay đổi, các message sẽ được giải mã bằng chính khóa đối xứng đó.

Nhờ vậy, trong record protocol, chúng ta đạt được cả confidentiality (tính bảo mật) và integrity (tính toàn vẹn).

Vì lượng dữ liệu được mã hóa trong giai đoạn này là rất lớn, nên quá trình này thường được gọi là bulk encryption.

Lúc này, bạn có thể thắc mắc: Tại sao TLS lại sử dụng cả mật mã đối xứng và bất đối xứng?

Tại sao không chỉ dùng một loại cho tất cả?

Rất dễ để thấy rằng mật mã đối xứng không thể cung cấp khả năng xác thực, bởi vì chỉ có một khóa bí mật duy nhất được chia sẻ giữa client và server. Hai bên không có cách nào để xác minh danh tính của nhau. Chưa kể đến việc làm sao cả hai có thể tạo ra cùng một khóa mà không bị lộ ra ngoài cũng là một vấn đề khó.

Vậy còn mật mã bất đối xứng thì sao? Nghe có vẻ là một ứng viên tốt.

Đáng tiếc là nó chậm hơn rất nhiều so với mật mã đối xứng. Và “rất nhiều” ở đây nghĩa là chậm hơn từ 100 lần đến thậm chí 10.000 lần.

Rõ ràng, mật mã bất đối xứng không phù hợp cho việc mã hóa dữ liệu khối lượng lớn (bulk encryption).

Được rồi, bây giờ hãy tìm hiểu kỹ hơn về mật mã đối xứng.

Có lẽ bạn đã biết những kiến thức cơ bản rồi.

Trước hết, Alice có một thông điệp dạng plaintext mà cô ấy muốn gửi cho Bob, nhưng không muốn bất kỳ ai trên mạng công cộng có thể đọc được.

Vì vậy, Alice sẽ mã hóa thông điệp bằng một khóa bí mật mà cô và Bob đã chia sẻ với nhau từ trước. Sau đó, cô gửi thông điệp đã được mã hóa cho Bob thông qua Internet công cộng.

Khi nhận được thông điệp đã mã hóa, Bob có thể dễ dàng sử dụng cùng khóa bí mật đó để giải mã.

Vì cùng một khóa được dùng cho cả mã hóa và giải mã, nên quá trình này mang tính đối xứng, và đó chính là lý do nó được gọi là mật mã đối xứng (symmetric cryptography).

Bây giờ, hãy tưởng tượng có một hacker tên là Harry, người có thể chặn được các message trao đổi của họ trên mạng công cộng. Tuy nhiên, vì message đã được mã hóa và Harry không có khóa bí mật, nên anh ta không thể giải mã nội dung.

Nhưng... anh ta vẫn có thể chỉnh sửa message đó!

BIT-FLIPPING ATTACK

Có một kỹ thuật tấn công gọi là bit-flipping attack, và nó hoạt động như sau.

Giả sử lần này Alice không nói chuyện với Bob, mà đang giao tiếp với ngân hàng trực tuyến của cô ấy. Alice muốn chuyển 100 đô la cho một người nào đó. Thông điệp được mã hóa bằng một khóa bí mật và gửi đến ngân hàng thông qua Internet.

Bây giờ, Harry chặn được thông điệp đã mã hóa. Mặc dù anh ta không thể giải mã nội dung, nhưng anh ta có thể lật một số bit trong thông điệp — từ 1 thành 0 hoặc từ 0 thành 1 — rồi chuyển tiếp thông điệp đã bị chỉnh sửa đó đến ngân hàng.

Khi ngân hàng giải mã thông điệp, họ sẽ nhận được nội dung plaintext khác. Trong trường hợp này, số tiền đã bị thay đổi thành 900 đô la thay vì 100 đô la.

Đây là một tình huống cực kỳ nguy hiểm. Chính vì vậy, chúng ta cần phải đảm bảo rằng thông điệp đã được mã hóa không bị thay đổi trong quá trình truyền tải.

Nhưng câu hỏi đặt ra là: làm thế nào để đảm bảo điều đó?

AUTHENTICATED ENCRYPTION

Một cách để giải quyết vấn đề này là sử dụng Authenticated Encryption (mã hóa có xác thực). Ý tưởng cốt lõi là không chỉ mã hóa, mà còn xác thực thông điệp đã được mã hóa.

Bước đầu tiên là mã hóa (encryption).

Thông điệp plaintext của Alice sẽ được đưa vào một thuật toán mã hóa đối xứng, chẳng hạn như AES-256-GCM hoặc ChaCha20.

Thuật toán mã hóa này nhận khóa bí mật dùng chung và một nonce ngẫu nhiên, hay còn gọi là Initialization Vector (IV), làm đầu vào. Kết quả trả về là thông điệp đã được mã hóa.

Bước thứ hai là xác thực (authentication).

Thông điệp đã được mã hóa, khóa bí mật, và nonce sẽ trở thành đầu vào của một thuật toán MAC.

Nếu bạn sử dụng AES-256-GCM, thuật toán MAC sẽ là GMAC.

Nếu bạn sử dụng ChaCha20, thuật toán MAC sẽ là Poly1305.

Thuật toán MAC này hoạt động tương tự như một hàm băm mật mã (cryptographic hash function), và đầu ra của nó là MAC (Message Authentication Code).

MAC này sẽ được đính kèm cùng với thông điệp đã mã hóa, và kết quả cuối cùng sẽ được gửi cho Bob. Vì vậy, đôi khi MAC này còn được gọi là authentication tag.

Trong TLS 1.3, ngoài việc xác thực thông điệp đã được mã hóa, chúng ta còn cần xác thực thêm một số dữ liệu liên quan (associated data).

Các dữ liệu này bao gồm những thông tin như địa chỉ, cổng (port), phiên bản giao thức, hoặc sequence number. Những thông tin này không được mã hóa và đã được cả hai bên biết trước.

Do đó, associated data cũng được đưa vào làm đầu vào của thuật toán MAC.

Chính vì vậy, toàn bộ quá trình này được gọi là Authenticated Encryption with Associated Data, hay viết tắt là AEAD.

DECRYPTION + VERIFICATION

Bây giờ, chúng ta sẽ xem Bob kiểm tra như thế nào để biết thông điệp đã được mã hóa có bị thay đổi trong quá trình truyền hay không.

Cách làm rất đơn giản — đó là quy trình ngược lại.

Bắt đầu với thông điệp đã mã hóa kèm theo MAC, Bob sẽ tách (untag) MAC ra khỏi thông điệp đã mã hóa.

Sau đó, thông điệp đã mã hóa sẽ được đưa vào thuật toán MAC, cùng với khóa bí mật dùng chung và nonce.

Lưu ý rằng nonce này chính là nonce đã được sử dụng trong quá trình mã hóa. Thông thường, nonce sẽ được đính kèm cùng thông điệp đã mã hóa trước khi gửi đến bên nhận.

Associated data cũng sẽ được đưa vào làm đầu vào của thuật toán MAC.

Đầu ra của thuật toán MAC lúc này sẽ là một MAC khác.

Bây giờ, Bob chỉ cần so sánh hai giá trị MAC:

- Nếu hai MAC khác nhau, Bob sẽ biết rằng thông điệp đã bị thay đổi trong quá trình truyền.
- Ngược lại, nếu hai MAC giống nhau, Bob có thể yên tâm giải mã thông điệp và sử dụng nó với sự tin chắc rằng đây chính là plaintext mà Alice đã gửi.

KEY EXCHANGE

Tuy nhiên, vẫn còn một câu hỏi: Làm thế nào Alice và Bob có thể chia sẻ khóa bí mật với nhau mà không làm lộ nó ra công khai?

Câu trả lời là: họ cần sử dụng mật mã bất đối xứng, hay còn gọi là public-key cryptography. Cụ thể, họ có thể dùng Diffie–Hellman Ephemeral (DHE) hoặc Elliptic-Curve Diffie–Hellman Ephemeral (ECDHE).

Được rồi, hãy cùng xem Diffie–Hellman key exchange hoạt động như thế nào.

Trước hết, Alice và Bob cùng thống nhất hai con số:

- g (base)
- p (modulus)

Hai con số này được công khai và ai cũng có thể biết.

Sau đó, mỗi người sẽ bí mật chọn một số riêng:

- Khóa riêng (private key) của Alice là số a
- Khóa riêng (private key) của Bob là số b

Tiếp theo, Alice tính khóa công khai của mình:

$$A = g^a \bmod p$$

và gửi A cho Bob.

Tương tự, Bob tính khóa công khai của mình:

$$B = g^b \bmod p$$

và gửi B cho Alice.

Lúc này:

- Alice nhận được khóa công khai B của Bob

- Bob nhận được khóa công khai A của Alice

Bây giờ, điều “kỳ diệu” xảy ra.

Alice tính:

$$S = B^a \bmod p$$

Bob tính:

$$S = A^b \bmod p$$

Và thật kỳ diệu, hai giá trị này lại bằng nhau, cùng bằng S.

Tại sao lại như vậy?

Bởi vì nếu làm toán, bạn sẽ thấy cả hai đều bằng: $g^{(a \times b)} \bmod p$

Nhờ đó, Alice và Bob tạo ra cùng một số bí mật S mà không hề làm lộ nó ra công khai.

Tuy nhiên, cần lưu ý rằng mỗi thuật toán mã hóa có thể yêu cầu độ dài khóa bí mật khác nhau. Vì vậy, để tạo ra khóa bí mật cuối cùng, Alice và Bob cần đưa S vào một hàm dẫn xuất khóa (Key Derivation Function – KDF).

Đầu ra của hàm này sẽ là khóa bí mật dùng chung với độ dài phù hợp cho thuật toán mã hóa.

Trong TLS 1.3, người ta sử dụng hàm dẫn xuất khóa dựa trên HMAC, và đó chính là lý do tại sao nó được gọi là HKDF.

KEY DERIVATION FUNCTION

Hãy cùng tìm hiểu kỹ hơn một chút về hàm dẫn xuất khóa (Key Derivation Function – KDF).

Nói chung, KDF nhận một đầu vào gọi là Input Key Material (IKM). Trong trường hợp của chúng ta, IKM chính là số S.

Chúng ta cũng cần chỉ rõ độ dài của khóa đầu ra mà mình mong muốn, ví dụ như 128-bit.

Ngoài ra, KDF còn cần một hàm băm mật mã, chẳng hạn như HMAC-SHA256.

Bên cạnh đó, KDF có thể (không bắt buộc) nhận thêm thông tin ngữ cảnh hoặc thông tin đặc thù của ứng dụng, cùng với một salt.

Với tất cả những đầu vào đó, KDF sẽ tạo ra một khóa bí mật có độ dài đúng như yêu cầu.

Bây giờ, quay trở lại với Diffie–Hellman key exchange.

IS DIFFIE-HELLMAN SECURED?

Vậy ta có thể đặt câu hỏi: Cơ chế trao đổi khóa này có an toàn không?

Hay là, với các giá trị p , g , A và B được tính theo những công thức này, Harry có thể suy ra các số bí mật a , b và S không?

May mắn là, những hàm này sẽ trở thành các hàm bẫy (trapdoor functions) nếu chúng ta chọn các giá trị p , g , a và b đủ tốt.

Ví dụ:

- Chọn p là một số nguyên tố 2048-bit
- Chọn g là một căn nguyên thủy modulo p
- Chọn a và b là các số ngẫu nhiên 256-bit

Hàm bẫy (trapdoor function) về cơ bản có nghĩa là: Dễ tính theo một chiều, nhưng rất khó tính theo chiều ngược lại.

Trong trường hợp này:

Với p , g và a , thì việc tính A là dễ

Nhưng với p , g và A , thì việc suy ra a là rất khó

Ta có thể thấy rõ rằng A có thể được tính rất nhanh, với độ phức tạp thời gian $O(\log(a))$. Đây là một bài toán lũy thừa modulo, vốn đã rất quen thuộc.

Ngược lại, việc tính a thì khó hơn rất nhiều.

Đây là bài toán logarit rời rạc (discrete logarithm problem) — một bài toán mà các máy tính hiện nay phải mất thời gian cực kỳ dài mới giải được.

Vì vậy, ít nhất là ở thời điểm hiện tại, chúng ta vẫn an toàn, hoặc cho đến khi thế hệ máy tính lượng tử mạnh mẽ tiếp theo xuất hiện.

Tuy nhiên, hiện tại thì,

“mất rất nhiều thời gian để giải” không có nghĩa là “không thể giải được”, đúng không?

STATIC KEY

Nếu Alice và Bob sử dụng cùng một khóa riêng, tức là a và b , cho mọi phiên giao tiếp, thì điều gì sẽ xảy ra?

Khi đó, Harry có thể ghi lại toàn bộ các phiên giao tiếp và bắt đầu cố gắng giải ra khóa riêng a từ dữ liệu của các phiên đó.

Mặc dù việc giải ra a sẽ mất rất nhiều thời gian, nhưng giả sử rằng đến phiên thứ N , hẳn cuối cùng cũng tìm được giá trị đúng của a .

Lúc này, Harry có thể dùng a để tính ra số bí mật S , và như vậy, hẳn sẽ có khả năng giải mã toàn bộ các cuộc trao đổi đã được ghi lại trước đó.

Nghe có đáng sợ không?

Vậy làm thế nào để ngăn chặn điều này?

EPHEMERAL KEY

Câu trả lời là khóa tạm thời (ephemeral key).

Đúng như tên gọi của nó, chúng ta sử dụng các khóa riêng khác nhau cho mỗi phiên giao tiếp.

Vì vậy, ngay cả khi Harry có thể giải được khóa bí mật của một phiên, hẳn cũng không thể dùng khóa đó để giải mã các phiên khác.

Cơ chế này được gọi là Perfect Forward Secrecy (PFS) trong TLS.

Như vậy, giờ bạn đã hiểu Diffie-Hellman Ephemeral nghĩa là gì rồi: đó đơn giản là Diffie-Hellman nhưng sử dụng các khóa tạm thời, tồn tại trong thời gian ngắn.

ELLIPTIC-CURVE CRYPTOGRAPHY

Vậy còn Elliptic-Curve Diffie-Hellman Ephemeral (ECDHE) thì sao?

Mật mã đường cong elliptic (Elliptic-Curve Cryptography – ECC) là một cách tiếp cận của mật mã bất đối xứng, trong đó thuật toán về cơ bản là tương tự, nhưng sử dụng một hàm bẫy (trapdoor function) khác.

Hàm bẫy này dựa trên cấu trúc đại số của các đường cong elliptic, và đó cũng chính là lý do cho cái tên này.

Một điểm cực kỳ ấn tượng của ECC là: nó chỉ cần khóa nhỏ hơn nhưng vẫn cung cấp mức độ bảo mật tương đương.

Bạn có thể thấy điều này trong bảng so sánh với RSA.
Cơ quan An ninh Quốc gia Hoa Kỳ (NSA) từng bảo vệ các thông tin tuyệt mật bằng khóa ECC 384-bit,
mà mức độ bảo mật tương đương với khóa RSA 7680-bit.

Nghe thật đáng kinh ngạc, đúng không?

Tuy nhiên, mật mã đường cong elliptic lại là mục tiêu dễ hơn đối với các cuộc tấn công bằng máy tính lượng tử.

Thuật toán Shor có thể phá vỡ ECC trên một máy tính lượng tử giả định, với ít tài nguyên lượng tử hơn so với việc phá RSA.

Có thể phải mất hàng chục năm nữa trước khi một máy tính lượng tử đủ mạnh thực sự được xây dựng và sử dụng, nhưng chúng ta đã chuẩn bị gì cho điều đó chưa?

Có thuật toán nào kháng lượng tử không? Có.
Đó là thuật toán trao đổi khóa Supersingular Isogeny Diffie-Hellman (SIDH), một thuật toán cũng dựa trên mật mã đường cong elliptic.

ASYMMETRIC CRYPTOGRAPHY

Nhưng đó lại là một câu chuyện khác.
Bây giờ, hãy quay lại với mật mã bất đối xứng.

Đây là một công nghệ cực kỳ mạnh mẽ, có rất nhiều ứng dụng khác nhau.

Chúng ta đã khám phá một ứng dụng của nó rồi, đó là trao đổi khóa bí mật đối xứng, thông qua Diffie-Hellman Ephemeral và Elliptic-Curve Diffie-Hellman Ephemeral.

Trên thực tế, thuật toán RSA cũng từng được dùng để trao đổi khóa trong quá khứ, nhưng đã bị loại bỏ trong TLS 1.3 do nhiều kiểu tấn công và không hỗ trợ forward secrecy.

Mật mã bất đối xứng cũng được sử dụng trong hệ thống mã hóa dữ liệu.

Dưới đây là một số thuật toán mã hóa bất đối xứng:

- RSA với Optimal Asymmetric Encryption Padding (OAEP)
- RSA với chuẩn PKCS#1 (Public Key Cryptography Standard 1)
- Thuật toán mã hóa ElGamal

Cuối cùng, một tính năng cực kỳ quan trọng khác của mật mã bất đối xứng là chữ ký số, mà TLS sử dụng rất nhiều cho mục đích xác thực.

Một số thuật toán chữ ký số phổ biến được dùng trong TLS gồm:

- RSA với Probabilistic Signature Scheme (RSA-PSS)
- Elliptic-Curve Digital Signature Algorithm (ECDSA)
- Edwards-Curve Digital Signature Algorithm (EdDSA)

Chúng ta sẽ tìm hiểu về chữ ký số ngay sau đây.

ASYMMETRIC ENCRYPTION

Nhưng trước khi đi vào đó, hãy cùng tìm hiểu hệ thống mã hóa bất đối xứng hoạt động như thế nào.

Tương tự như trong mã hóa đối xứng, Alice có một bản rõ (plaintext) mà cô ấy muốn gửi cho Bob.

Nhưng lần này, không có khóa bí mật được chia sẻ trước.

Thay vào đó, Alice mã hóa thông điệp bằng khóa công khai (public key) của Bob, rồi gửi thông điệp đã được mã hóa cho Bob.

Khi Bob nhận được thông điệp, anh ấy sử dụng khóa riêng (private key) của mình để giải mã.

Mặc dù khóa công khai và khóa riêng là khác nhau, chúng vẫn được liên kết với nhau thông qua một hàm bẫy (trapdoor function), giống như những gì chúng ta đã thấy trong thuật toán Diffie-Hellman.

Ý tưởng cốt lõi là:

- Khóa luôn tồn tại theo cặp
- Chỉ khóa riêng của cặp đó mới có thể giải mã dữ liệu được mã hóa bằng khóa công khai tương ứng

Chính vì vậy, ngay cả khi hacker Harry có quyền truy cập vào thông điệp đã mã hóa của Alice và khóa công khai của Bob, hắn cũng không thể dùng khóa công khai đó để giải mã thông điệp.

PUBLIC KEY SHARING

Vì vậy, việc chia sẻ khóa công khai trở nên rất đơn giản.

Bob chỉ cần gửi khóa công khai của mình cho Alice trực tiếp qua Internet công cộng, mà không cần lo sợ khóa đó sẽ bị dùng để giải mã bất kỳ thông điệp nào.

Khóa này là khóa công khai, nên bất kỳ ai cũng có thể dùng nó để mã hóa thông điệp mà chỉ Bob mới có thể đọc được, ngay cả khi họ chưa từng liên lạc với nhau trước đó.

Điều này thực sự rất kỳ diệu, đúng không?

Tuy nhiên, mọi chuyện không đơn giản đến thế.

Mặc dù chúng ta biết rằng Harry không thể giải mã thông điệp bằng khóa công khai của Bob,

MIDDLE MAN REPLACES KEY

, hắn vẫn có thể can thiệp vào quá trình chia sẻ khóa công khai, và thay thế khóa công khai của Bob bằng khóa công khai của chính hắn.

Khi đó, Alice nhận được khóa và vẫn nghĩ rằng đó là khóa công khai của Bob, nhưng thực chất lại là khóa công khai của Harry.

Vì vậy, nếu Alice mã hóa thông điệp của mình bằng khóa này, thì Harry có thể giải mã nó bằng khóa riêng của hắn.

Lý do khiến điều này có thể xảy ra là vì một khóa chỉ đơn thuần là một con số, và không có bất kỳ thông tin định danh nào để cho biết chủ sở hữu của nó là ai.

Vậy chúng ta có thể làm gì?

Rõ ràng, chúng ta cần gắn khóa công khai với một số thông tin định danh.

DIGITAL CERTIFICATE

Và đó chính là chứng chỉ số (digital certificate), không gì khác cả.

Vì vậy, Bob đặt khóa công khai của mình vào trong chứng chỉ số, trong đó có tên của Bob và các thông tin định danh khác.

Chứng chỉ số hoạt động giống như hộ chiếu ngoài đời thực.

Nhưng làm sao chúng ta biết chứng chỉ đó thực sự thuộc về Bob?

Điều gì ngăn cản Harry tạo ra một chứng chỉ giả mang tên Bob nhưng lại chứa khóa công khai của Harry?

Câu trả lời là: giống như ngoài đời thực, hộ chiếu phải được cấp bởi cơ quan cấp hộ chiếu sau khi xác minh danh tính.

Trong thế giới số, chứng chỉ phải được xác minh và ký bởi một tổ chức cấp chứng chỉ – Certificate Authority (CA).

Certificate Authority trong thế giới số và cơ quan cấp hộ chiếu ngoài đời thực đều là bên thứ ba đáng tin cậy, giúp chúng ta ngăn chặn việc tạo ra hộ chiếu và chứng chỉ số giả mạo.

CERTIFICATE SIGNING

Quy trình ký chứng chỉ diễn ra như sau:

Bob có một cặp khóa công khai và khóa riêng.

Bước đầu tiên, Bob tạo ra một yêu cầu ký chứng chỉ (Certificate Signing Request – CSR).

CSR này chứa khóa công khai của Bob và một số thông tin định danh, chẳng hạn như tên, tổ chức và địa chỉ email.

Bước thứ hai, Bob ký CSR bằng khóa riêng của mình, rồi gửi CSR đó cho tổ chức cấp chứng chỉ (Certificate Authority – CA).

CA sẽ xác minh danh tính của Bob trong chứng chỉ.

Họ có thể liên hệ với Bob để yêu cầu thêm bằng chứng nếu cần.

Sau đó, CA sử dụng khóa công khai của Bob trong chứng chỉ để xác minh chữ ký. Mục đích là để đảm bảo rằng Bob thực sự sở hữu khóa riêng tương ứng với khóa công khai trong chứng chỉ.

Nếu mọi thứ đều hợp lệ, CA sẽ ký chứng chỉ bằng khóa riêng của chính họ, và gửi chứng chỉ đã ký đó lại cho Bob.

Bây giờ, Bob sẽ chia sẻ chứng chỉ này với Alice, chứng chỉ có chứa khóa công khai của Bob, thay vì chỉ gửi mỗi khóa công khai như trước.

Khi nhận được chứng chỉ, Alice có thể dễ dàng xác minh tính xác thực của nó bằng khóa công khai của CA.

Nhờ vậy, Harry không thể thay thế khóa công khai của Bob bằng khóa của hắn nữa, bởi vì hắn không có khóa riêng của CA để ký một chứng chỉ giả mạo.

Lưu ý rằng, cơ chế này chỉ hoạt động vì tất cả chúng ta đều tin tưởng Certificate Authority.

Nếu vì lý do nào đó CA không đáng tin cậy, ví dụ như họ đưa khóa riêng của mình cho Harry, thì chúng ta sẽ gặp rắc rối cực kỳ nghiêm trọng!

A CHAIN OF TRUST

Trong thực tế, tồn tại một chuỗi các Certificate Authority (CA). Ở cấp cao nhất là Root Certificate Authority (CA gốc).

CA gốc tự ký chứng chỉ của chính mình, và đồng thời ký chứng chỉ cho các CA cấp dưới, những CA này được gọi là Intermediate Certificate Authority (CA trung gian).

Các CA trung gian có thể ký chứng chỉ cho các CA trung gian khác, hoặc ký trực tiếp cho chứng chỉ của thực thể cuối (end-entity certificate hay còn gọi là leaf certificate).

Mỗi chứng chỉ đều tham chiếu ngược về chứng chỉ của CA cấp cao hơn, và chuỗi này kéo dài cho đến CA gốc.

Hệ điều hành và trình duyệt của bạn lưu sẵn danh sách các chứng chỉ của những CA gốc đáng tin cậy.

Nhờ vậy, chúng có thể dễ dàng xác minh tính xác thực của mọi chứng chỉ.

Giờ hãy cùng xem một chứng chỉ TLS thực tế của YouTube.

Trên Chrome, chúng ta nhấp vào biểu tượng ổ khóa và chọn Certificate.

Đây là chứng chỉ của thực thể cuối (end-entity certificate).

Nó được cấp bởi Google Trust Services (GTS) và thuật toán chữ ký được sử dụng là RSA với hàm băm SHA-256.

Khóa công khai của chứng chỉ sử dụng mật mã đường cong elliptic (Elliptic Curve Cryptography) với độ dài khóa 256 bit, nên khóa trông khá ngắn.

Đây là chữ ký của chứng chỉ, được GTS ký.

Khi cuộn xuống một chút, chúng ta có thể thấy rằng chứng chỉ này được dùng cho nhiều website của Google, bao gồm youtube.com.

Chứng chỉ này sẽ hết hạn vào ngày 26 tháng 5 năm 2020.

Bây giờ, hãy xem chứng chỉ của CA đã ký chứng chỉ này.

Đó là một CA trung gian, và tên của nó cũng là Google Trust Services.

CA này cũng có khóa công khai, nhưng thuộc loại khác: mã hóa RSA.

Vì vậy, độ dài khóa lớn hơn nhiều, lên tới 2048 bit.

Đây là chữ ký của CA trung gian, được CA gốc ký.

CA gốc ở đây là GlobalSign.

Và đây là khóa công khai RSA của GlobalSign, cùng với chữ ký tự ký (self-signed) của chính nó.

DIGITAL SIGNATURE

Chúng ta đã nói khá nhiều về chữ ký số, vậy bây giờ hãy cùng xem nó thực sự hoạt động như thế nào!

Để ký một tài liệu, người ký trước hết cần băm (hash) tài liệu đó.

Sau đó, giá trị băm này được mã hóa bằng khóa riêng của người ký.

Kết quả của bước này chính là chữ ký số.

Tiếp theo, chữ ký số được đính kèm vào tài liệu gốc.

Và như vậy là quá trình ký đã hoàn tất.

Vậy làm thế nào để xác minh chữ ký có hợp lệ hay không?

Rất đơn giản, ta chỉ cần thực hiện quá trình ngược lại.

Đầu tiên, tách chữ ký ra khỏi tài liệu.

Sau đó, giải mã chữ ký bằng khóa công khai của người ký để thu được một giá trị băm.

Tiếp theo, băm lại tài liệu bằng cùng thuật toán băm đã được sử dụng khi ký.

Kết quả sẽ là một giá trị băm thứ hai.

Cuối cùng, so sánh hai giá trị băm này.

Nếu chúng giống nhau, thì chữ ký là hợp lệ.

OK, vậy với tất cả những kiến thức mà chúng ta đã học được cho đến giờ, hãy xem kỹ hơn cách chúng được sử dụng trong giao thức bắt tay TLS (TLS handshake).

TLS 1.3 FULL HANDSHAKE

Quá trình full handshake của TLS 1.3 bắt đầu bằng một thông điệp Hello mà client gửi cho server.

Thực ra thông điệp này chứa rất nhiều thông tin, nhưng ở đây ta chỉ liệt kê những thông tin quan trọng.

Trước hết là danh sách các phiên bản giao thức mà client hỗ trợ.

Tiếp theo là danh sách các bộ mã hóa đối xứng AEAD được hỗ trợ.

Trong trường hợp này có 2 lựa chọn: AES-256-GCM hoặc CHACHA20-POLY1305.

Sau đó là danh sách các nhóm trao đổi khóa được hỗ trợ.

Ví dụ, client này hỗ trợ cả Diffie-Hellman Ephemeral trên trường hữu hạn và Elliptic-Curve Diffie-Hellman Ephemeral.

Vì vậy, client cũng chia sẻ 2 khóa công khai: một cho Diffie-Hellman, và một cho Elliptic-Curve Diffie-Hellman.

Bằng cách này, server sẽ có thể tính toán khóa bí mật chung bất kể nó chọn thuật toán nào.

Trường cuối cùng mà client gửi trong thông điệp này là danh sách các thuật toán chữ ký mà client hỗ trợ.

Thông tin này dùng để server chọn thuật toán mà nó sẽ dùng để ký toàn bộ quá trình handshake.

Chúng ta sẽ thấy cách nó hoạt động ngay sau đây.

Sau khi nhận được Client Hello, server gửi lại thông điệp Hello của mình.

Thông điệp này chứa:

- Phiên bản giao thức được chọn: TLS 1.3

- Bộ mã hóa được chọn: AES-256-GCM
- Phương thức trao đổi khóa được chọn: Diffie-Hellman Ephemeral
- Khóa công khai của server cho phương thức đã chọn

Trường tiếp theo là yêu cầu chứng chỉ của client, trường này là tùy chọn và chỉ được gửi nếu server muốn xác thực client bằng chứng chỉ.

Thông thường, trên các website HTTPS, chỉ phía server cần gửi chứng chỉ cho client.

Và chứng chỉ đó được gửi trong trường tiếp theo của thông điệp này.

Trường tiếp theo là Certificate Verify, thực chất chính là chữ ký của toàn bộ quá trình handshake cho đến thời điểm này.

Cách tạo ra chữ ký này như sau:

Toàn bộ dữ liệu từ lúc bắt đầu handshake cho đến Certificate Request được gọi là handshake context.

Chúng ta nối (concatenate) context này với chứng chỉ của server, băm (hash) chúng, rồi ký giá trị băm bằng khóa riêng của server, sử dụng một trong các thuật toán chữ ký mà client hỗ trợ.

Theo cách tương tự, Server Finished được tạo ra bằng cách:

Nối handshake context, chứng chỉ, và certificate verify, sau đó băm, và đưa giá trị băm vào thuật toán MAC của bộ mã hóa đã chọn.

Kết quả là MAC của toàn bộ quá trình handshake.

Ở đây, server certificate, certificate verify và server finished được gọi là các thông điệp xác thực, vì chúng được dùng để xác thực server.

Nhờ chữ ký và MAC của toàn bộ handshake, TLS 1.3 an toàn trước nhiều loại tấn công man-in-the-middle và downgrade.

Sau khi client nhận được Hello từ server, nó sẽ xác minh chứng chỉ của server với CA gốc, và kiểm tra chữ ký cũng như MAC của toàn bộ handshake để đảm bảo không bị chỉnh sửa trong quá trình truyền.

Nếu mọi thứ đều ổn, client sẽ gửi thông điệp Finished của mình, chứa MAC của toàn bộ handshake cho đến thời điểm đó.

Ngoài ra, client có thể gửi kèm chứng chỉ và certificate verify của mình, nếu server đã yêu cầu.

Và đó chính là toàn bộ luồng của một full TLS handshake.

Để cải thiện hiệu năng, client và server không phải lúc nào cũng thực hiện full handshake.

Đôi khi, họ sẽ thực hiện handshake rút gọn (abbreviated handshake).

RESUMPTION & PRE-SHARED KEY

Điều này được thực hiện bằng cách resume phiên với khóa dùng chung sẵn (Pre-Shared Key – PSK).

Ý tưởng là: sau lần handshake trước, client và server đã biết nhau rồi, nên không cần xác thực lại.

Vì vậy, server có thể gửi một hoặc nhiều session ticket cho client.

Những ticket này có thể được dùng làm danh tính PSK trong lần handshake tiếp theo.

Mỗi ticket đi kèm với thời hạn hiệu lực cũng như một số thông tin khác.

Trong lần handshake kế tiếp, client sẽ gửi một thông điệp Hello đơn giản.

Thông điệp này chứa:

- Danh sách các PSK identity (hoặc ticket) nhận được từ lần handshake trước
- Chế độ trao đổi khóa PSK, có thể là: PSK only, hoặc PSK kết hợp với Diffie-Hellman

Nếu chế độ PSK + Diffie-Hellman được sử dụng, thì client cũng cần chia sẻ khóa công khai Diffie-Hellman của mình.

Cách này vừa cung cấp perfect forward secrecy, vừa cho phép server quay lại full handshake nếu cần.

Khi server nhận được Client Hello, nó sẽ gửi lại Server Hello với:

- PSK identity được chọn
- Khóa công khai Diffie-Hellman của server (tùy chọn)
- Server Finished, giống như trong full handshake

Cuối cùng, client gửi lại thông điệp Finished của mình.

Và đó chính là kết thúc quá trình PSK resumption.

Như bạn thấy, không có bước xác thực bằng chứng chỉ giữa client và server trong handshake rút gọn này.

ZERO ROUND-TRIP TIME (0-RTT)

VCơ chế này cũng mở ra cơ hội cho dữ liệu 0 round-trip time (0-RTT).

Điều này có nghĩa là client không cần chờ quá trình handshake hoàn tất mà có thể gửi dữ liệu ứng dụng đầu tiên cho server ngay lập tức.

Trong 0-RTT, client gửi dữ liệu ứng dụng cùng với thông điệp Client Hello.

Dữ liệu này được mã hóa bằng khóa được suy ra từ PSK đầu tiên trong danh sách ticket.

Ngoài ra, client thêm một trường nữa gọi là “early data indication”, để báo cho server biết rằng đang có dữ liệu ứng dụng sớm được gửi kèm.

Nếu server chấp nhận yêu cầu 0-RTT này, nó sẽ gửi lại Server Hello, giống như trong PSK resumption thông thường, và có thể kèm theo một số dữ liệu ứng dụng.

Client sau đó kết thúc bằng một thông điệp chứa MAC, cùng với chỉ báo kết thúc dữ liệu sớm (end-of-early-data indicator).

Đó chính là cách 0 round-trip time hoạt động trong TLS 1.3.

Ưu điểm của 0-RTT là giảm độ trễ đi 1 vòng round-trip.

Tuy nhiên, nhược điểm là nó mở ra nguy cơ tấn công replay.

Điều này có nghĩa là kẻ tấn công có thể sao chép và gửi lại cùng một yêu cầu 0-RTT đã được mã hóa nhiều lần tới server.

Để tránh điều này, ứng dụng phía server phải được thiết kế sao cho có khả năng chịu được các yêu cầu trùng lặp.

TLS 1.3 vs TLS 1.2

Giờ trước khi kết thúc, hãy so sánh nhanh TLS 1.3 và TLS 1.2 để xem điểm mới là gì.

Thứ nhất, TLS 1.3 có cơ chế trao đổi khóa an toàn hơn.

Các phương thức dễ bị tấn công như RSA và các cơ chế trao đổi khóa tĩnh khác đã bị loại bỏ, chỉ giữ lại Diffie-Hellman Ephemeral hoặc Elliptic-Curve Diffie-Hellman Ephemeral.

Nhờ đó, perfect forward secrecy được đảm bảo.

Thứ hai, quá trình handshake của TLS 1.3 nhanh hơn ít nhất 1 round-trip so với TLS 1.2.

Mã hóa đối xứng trong TLS 1.3 an toàn hơn, vì AEAD cipher suite là bắt buộc.

TLS 1.3 cũng loại bỏ nhiều thuật toán yếu, chẳng hạn như Block Cipher Mode, RC4 hoặc Triple DES.

Cipher suite trong TLS 1.3 đơn giản hơn, vì chỉ bao gồm thuật toán AEAD và một thuật toán băm.

Các thuật toán trao đổi khóa và chữ ký được tách ra thành các trường riêng biệt.

Trong khi đó, ở TLS 1.2, chúng được gộp chung vào cipher suite.

Như trong ví dụ này, DHE là thuật toán trao đổi khóa, và RSA là thuật toán chữ ký.

Cách thiết kế này khiến số lượng cipher suite được khuyến nghị trở nên rất lớn, trong khi ở TLS 1.3 chỉ còn 5 cipher suite.

Tiếp theo, TLS 1.3 còn cung cấp chữ ký mạnh hơn, vì nó ký toàn bộ quá trình handshake, chứ không chỉ một phần của handshake như TLS 1.2.

Cuối cùng nhưng không kém phần quan trọng, mật mã đường cong elliptic (ECC) được chú trọng hơn rất nhiều trong TLS 1.3.

Một số thuật toán đường cong tốt hơn đã được bổ sung, chẳng hạn như Edwards-Curve Digital Signature Algorithm, vốn nhanh hơn mà không làm giảm mức độ bảo mật.

Và đó là tất cả những gì tôi muốn chia sẻ với bạn trong video này. Cảm ơn bạn đã theo dõi, hẹn gặp lại trong video tiếp theo!